

Jenkins challenges

Use the below code and implement our CI/CD stages.

- **Source Code:** <https://github.com/betawins/hiring-app.git>
 - Repository contains the application code for the hiring app.
- Create one Declarative pipeline job
- Create one Scripted pipeline job
- Create one multi stage pipeline job
- Create one parallel stage pipeline job



CI/CD Implementation Document

Project: Hiring App

Repository: <https://github.com/betawins/hiring-app.git>

—->Objective

Implement CI/CD pipeline for the Hiring App with the following:

CI Stages

- Git Clone
- Maven Build
- SonarQube Analysis
- Nexus Artifact Upload

CD Stages

- Deploy to Tomcat

Post Build

- Slack Notification

Pipelines to be created:

- Declarative Pipeline

- Scripted Pipeline
- Multi-Stage Pipeline
- Parallel Stage Pipeline

—> Prerequisites

2.1 Jenkins Server

- Jenkins installed and running
- Java installed
- Port 8080 accessible

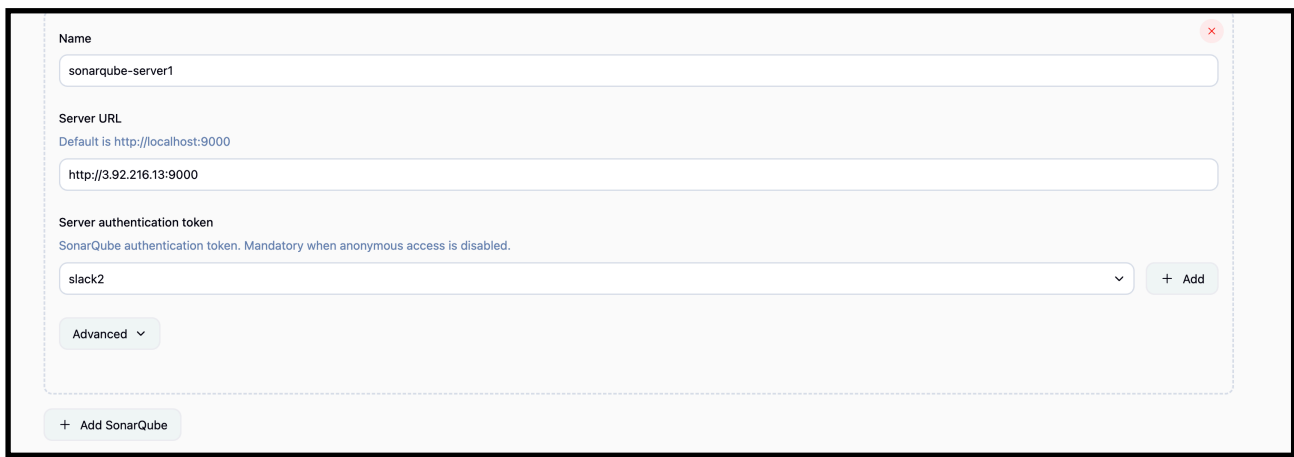
2.2 Tools Configuration (Jenkins → Manage Jenkins → Global Tool Configuration)

Tool	Name
Git	Default
Maven	MAVEN
Sonar Scanner	SonarScanner

—> SonarQube Setup

Steps

1. Start SonarQube server
2. Login → Admin → Security → Tokens
3. Generate token
4. Jenkins → Manage Jenkins → Configure System
5. Add SonarQube server:
 - Name: sonarqube-server1
 - Server URL: `http://<SONAR-IP>:9000`
 - Authentication token: Sonar token / (slack2 named token I have created)

A screenshot of the SonarQube configuration interface. It features a 'Name' field with 'sonarqube-server1', a 'Server URL' field with 'http://3.92.216.13:9000', and a 'Server authentication token' field with 'slack2'. There is an 'Advanced' dropdown menu and a '+ Add' button at the bottom right. A '+ Add SonarQube' button is located at the bottom left.

Name
sonarqube-server1

Server URL
Default is http://localhost:9000
http://3.92.216.13:9000

Server authentication token
SonarQube authentication token. Mandatory when anonymous access is disabled.
slack2

Advanced

+ Add SonarQube

4 Nexus Setup

Steps

1. Login to Nexus (`http://<NEXUS-IP>:8081`)

Create Maven repository:

- Type: Hosted
- Name: hiring-app

Create a credential for nexus

A screenshot of the Nexus Credentials interface. It shows a 'Credentials' dropdown menu with 'admin/***** (nexus-user)' selected. There is a '+ Add' button at the bottom right.

Credentials
admin/***** (nexus-user)

+ Add

—> Tomcat Setup

Steps

1. Install Tomcat
2. Enable manager-script role in `tomcat-users.xml`
3. Restart Tomcat
4. Verify URL:

`http://<TOMCAT-IP>:8080/manager/text`

5.

6. Jenkins → Credentials:

- ID: `tomcat_credentials`
- Username & Password

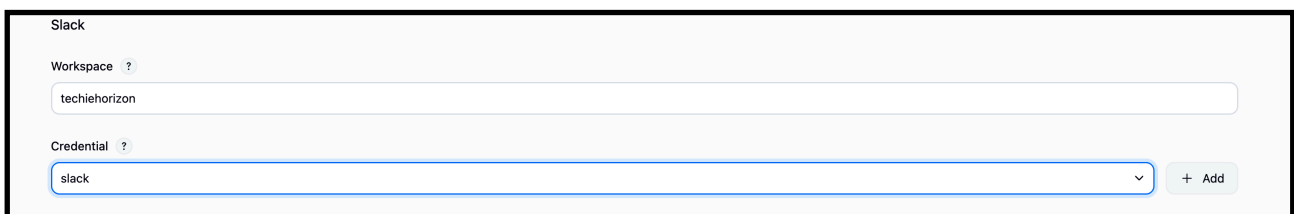


The screenshot shows the 'Tomcat 9.x Remote' configuration page in Jenkins. It features a 'Credentials' dropdown menu with the selected entry 'deployer/***** (tomcat_credentials)'. Below this is a 'Tomcat URL' field with a help icon.

—> Slack Integration

Steps

1. Create Slack Incoming Webhook
2. Copy Webhook URL
3. Jenkins → Manage Jenkins → Credentials



The screenshot shows the 'Slack' configuration page in Jenkins. It includes a 'Workspace' field with the value 'techiehorizon' and a 'Credential' dropdown menu with 'slack' selected. An '+ Add' button is visible next to the credential dropdown.

1.Declarative Pipeline Job

Steps

1. Jenkins Dashboard → New Item
2. Job Name: `hiring-app-declarative`
3. Select **Pipeline**
4. Definition: Pipeline script
5. Paste Declarative Jenkinsfile
6. Save → Build Now

Common Pipeline Stages (Used in All Jobs)

Stage	Purpose
Git Clone	Fetch application source code
Maven Build	Compile & package application
SonarQube	Code quality analysis
Nexus	Upload build artifact
Tomcat	Deploy application
Slack	Notify build status

Step :

1. Jenkins Dashboard → New Item
2. Job Name: `hiring-app-declarative`
3. Select **Pipeline**

Jenkins / All / New Item

New Item

Enter an item name

hiring-app-declarative

Select an item type

**Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

**Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

**Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

4. Paste Declarative Jenkinsfile

← → ↺ ⚠ Not Secure 3.235.20.129:8081/job/hiring-app-declarative/configure

Jenkins / hiring-app-declarative / Configuration

Configure

 General

 Triggers

 Pipeline

 Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   environment {
5     MAVEN_HOME = tool 'MVN_HOME'
6     SONAR_HOME = tool 'sonar_scanner'
7
8     SONAR_PROJECT_KEY = 'Sabear'
9     SONAR_PROJECT_NAME = 'Sabear'
10
11    NEXUS_URL = 'http://54.204.233.63:8081'
12    NEXUS_REPO = 'hiring-app'
13    NEXUS_CRED = 'nexus-user'
14
15    TOMCAT_URL = 'http://98.92.64.216:8080'
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

```

pipeline {
    agent any

    environment {
        MAVEN_HOME = tool 'MVN_HOME'
        SONAR_HOME = tool 'sonar_scanner'

        SONAR_PROJECT_KEY = 'Sabear'
        SONAR_PROJECT_NAME = 'Sabear'

        NEXUS_URL = 'http://54.204.233.63:8081'
        NEXUS_REPO = 'hiring-app'
        NEXUS_CRED = 'nexus-user'

        TOMCAT_URL = 'http://98.92.64.216:8080'
        TOMCAT_CONTEXT = 'simplecustomerapp'
        TOMCAT_CRED = 'tomcat_credentials'
    }

    stages {

        stage('Git Clone') {
            steps {
                git 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
            }
        }

        stage('Maven Build') {
            steps {
                sh """
                    ${MAVEN_HOME}/bin/mvn clean package
                    -Dmaven.test.failure.ignore=true
                """
            }
        }
    }
}

```

```

        """
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('sonarqube-server1') {
            sh """
                ${SONAR_HOME}/bin/sonar-scanner \
                -Dsonar.projectKey=$
{SONAR_PROJECT_KEY} \
                -Dsonar.projectName=$
{SONAR_PROJECT_NAME} \
                -Dsonar.sources=src \
                -Dsonar.java.binaries=target/**/WEB-INF/
classes
            """
        }
    }
}

stage('Upload WAR to Nexus') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: "${NEXUS_CRED}",
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASS'
        )]) {
            sh """
                WAR_FILE=$(ls target/*.war | head -n 1)

                curl -u \${NEXUS_USER}:\${NEXUS_PASS} \
                --upload-file \${WAR_FILE} \

```

```
        ${NEXUS_URL}/repository/${NEXUS_REPO}/in/javahome/hiring/0.1/hiring-0.1.war
    """
```

```
    }
  }
}
```

```
stage('Deploy to Tomcat') {
```

```
  steps {
```

```
    withCredentials([usernamePassword(
      credentialsId: "${TOMCAT_CRED}",
      usernameVariable: 'TOMCAT_USER',
      passwordVariable: 'TOMCAT_PASS'
```

```
    )]) {
```

```
      sh """
```

```
        WAR_FILE=$(ls target/*.war | head -n 1)
```

```
        curl -u \${TOMCAT_USER}:\${TOMCAT_PASS}
```

```
 \
```

```
        -T \${WAR_FILE} \
```

```
        "${TOMCAT_URL}/manager/text/deploy?
```

```
path=/\${TOMCAT_CONTEXT}&update=true"
```

```
      """
```

```
    }
```

```
  }
```

```
}
```

```
/* VISIBLE SLACK STAGE */
```

```
stage('Slack Notification') {
```

```
  steps {
```

```
    script {
```

```
      if (currentBuild.currentResult == 'SUCCESS') {
```

```
        slackSend(
```

```

        channel: '#jenkins-integration',
        color: 'good',
        message: "SUCCESS: ${JOB_NAME} #${BUILD_NUMBER} deployed to Tomcat"
    )
} else {
    slackSend(
        channel: '#jenkins-integration',
        color: 'danger',
        message: " FAILED: ${JOB_NAME} #${BUILD_NUMBER}"
    )
}
}
}
}
}

post {
    success {
        echo "Pipeline completed successfully"
    }
    failure {
        echo "Pipeline failed"
    }
}
}

```

--> save the groovy code

--> now click build now .

The screenshot shows the Jenkins web interface for a job named "hiring-app-declarative". The browser address bar indicates the URL is "3.235.20.129:8081/job/hiring-app-declarative/" and the connection is "Not Secure".

On the left sidebar, the "Status" tab is selected. Below it, there are links for "Changes", "Build Now", "Configure", "Delete Pipeline", "SonarQube", "Stages", "Rename", and "Pipeline Syntax".

The main content area shows the job name "hiring-app-declarative" with a green checkmark icon. Below this, the "SonarQube Quality Gate" section displays two status boxes: "Sabear Passed" and "server-side processing: Success".

Under the "Permalinks" section, there is a list of links for various build types and their completion times:

- [Last build \(#11\), 7 min 27 sec ago](#)
- [Last stable build \(#11\), 7 min 27 sec ago](#)
- [Last successful build \(#11\), 7 min 27 sec ago](#)
- [Last failed build \(#8\), 19 min ago](#)
- [Last unsuccessful build \(#8\), 19 min ago](#)
- [Last completed build \(#11\), 7 min 27 sec ago](#)

At the bottom left, there is a "Builds" section with a search filter. It lists the following builds:

Build Number	Time	Status
#11	10:29 AM	Success
#10	10:27 AM	Success
#9	10:20 AM	Success
#8	10:17 AM	Failed

--> when the build is passed we have to check the

← → ↻ ⚠ Not Secure 3.235.20.129:8081/job/hiring-app-declarative/11/console

Jenkins / hiring-app-declarative / #11 / Console Output

Status </> Changes Console Output Edit Build Information Delete build '#11' Timings Git Build Data Pipeline Overview Restart from Stage Replay Pipeline Steps Workspaces Previous Build

Console Output

```
Started by user waseem akram
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/hiring-app-declarative
[Pipeline] {
[Pipeline] tool
[Pipeline] tool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/hiring-app-declarative/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/sabear_simplecutomerapp.git # timeout=10
Fetching upstream changes from https://github.com/Waseema761/sabear_simplecutomerapp.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/sabear_simplecutomerapp.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision f386c3ea540af6aff21dd0c7f2c55de7d31d5219 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f f386c3ea540af6aff21dd0c7f2c55de7d31d5219 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
```

—> check the pipeline overview

← → ↻ ⚠ Not Secure 3.235.20.129:8081/job/hiring-app-declarative/11/pipeline-overview/

Jenkins / hiring-app-declarative / #11 / Pipeline Overview

< #11 Rerun

Manually run by waseem akram Started 1 hr 36 min ago Queued 2 ms Took 11 sec

Graph

Start Git Clone Maven Build SonarQube Analysis Upload WAR to Nexus Deploy to Tomcat Slack Notification Post Actions End

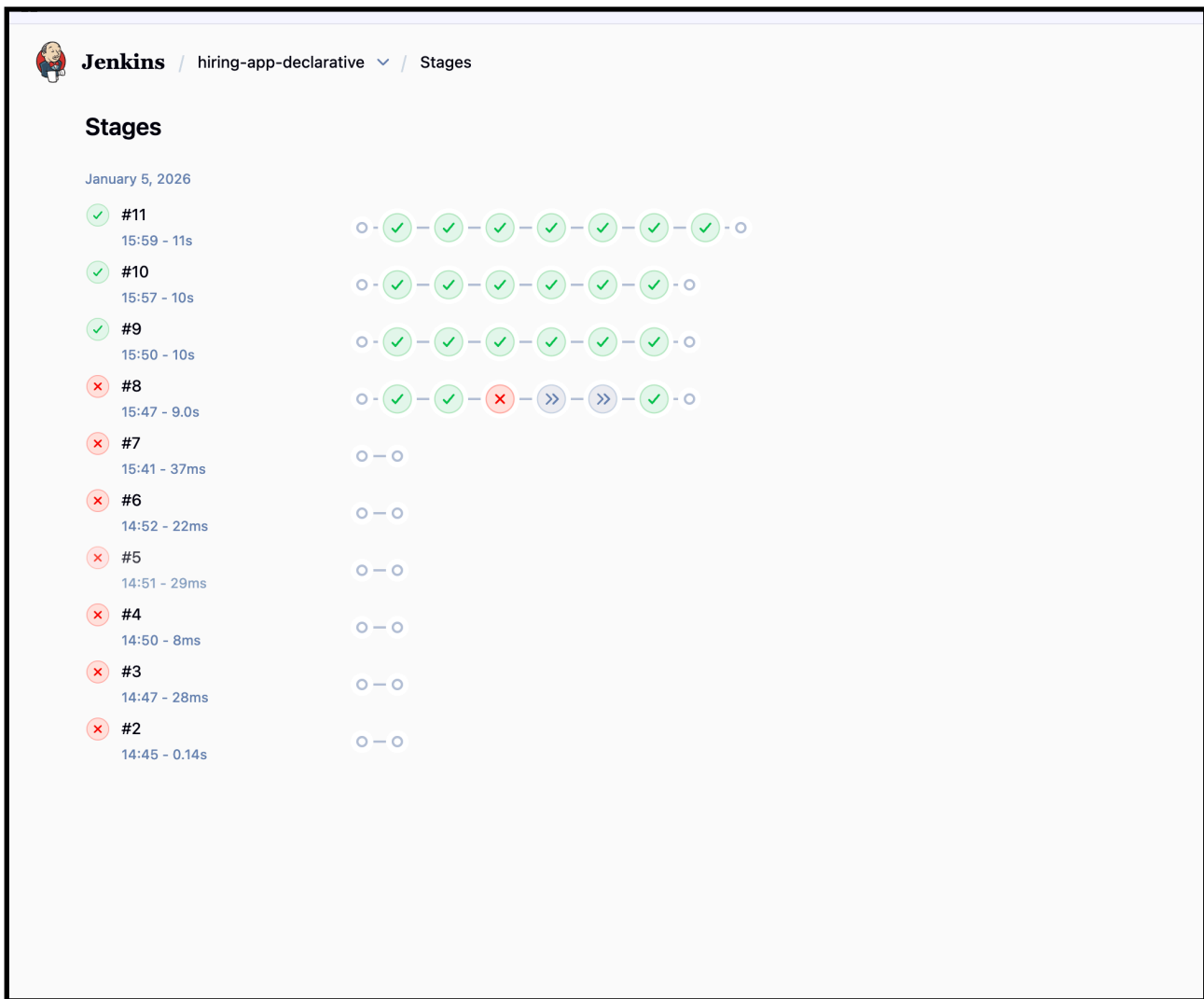
Search

Git Clone 0.17s Maven Build 2.9s SonarQube Analysis 6.6s Upload WAR to Nexus 0.35s Deploy to Tomcat 0.34s Slack Notification 0.14s Post Actions 40ms

Post Actions 40ms Started 1h 36m ago Jenkins

Pipeline completed successfully 15ms

0 Pipeline completed successfully



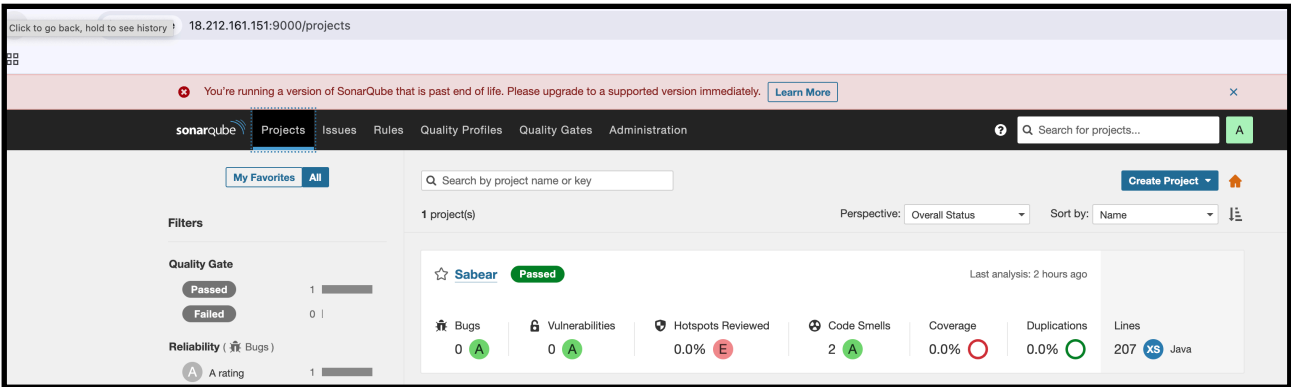
Validation Checklist

Git clone

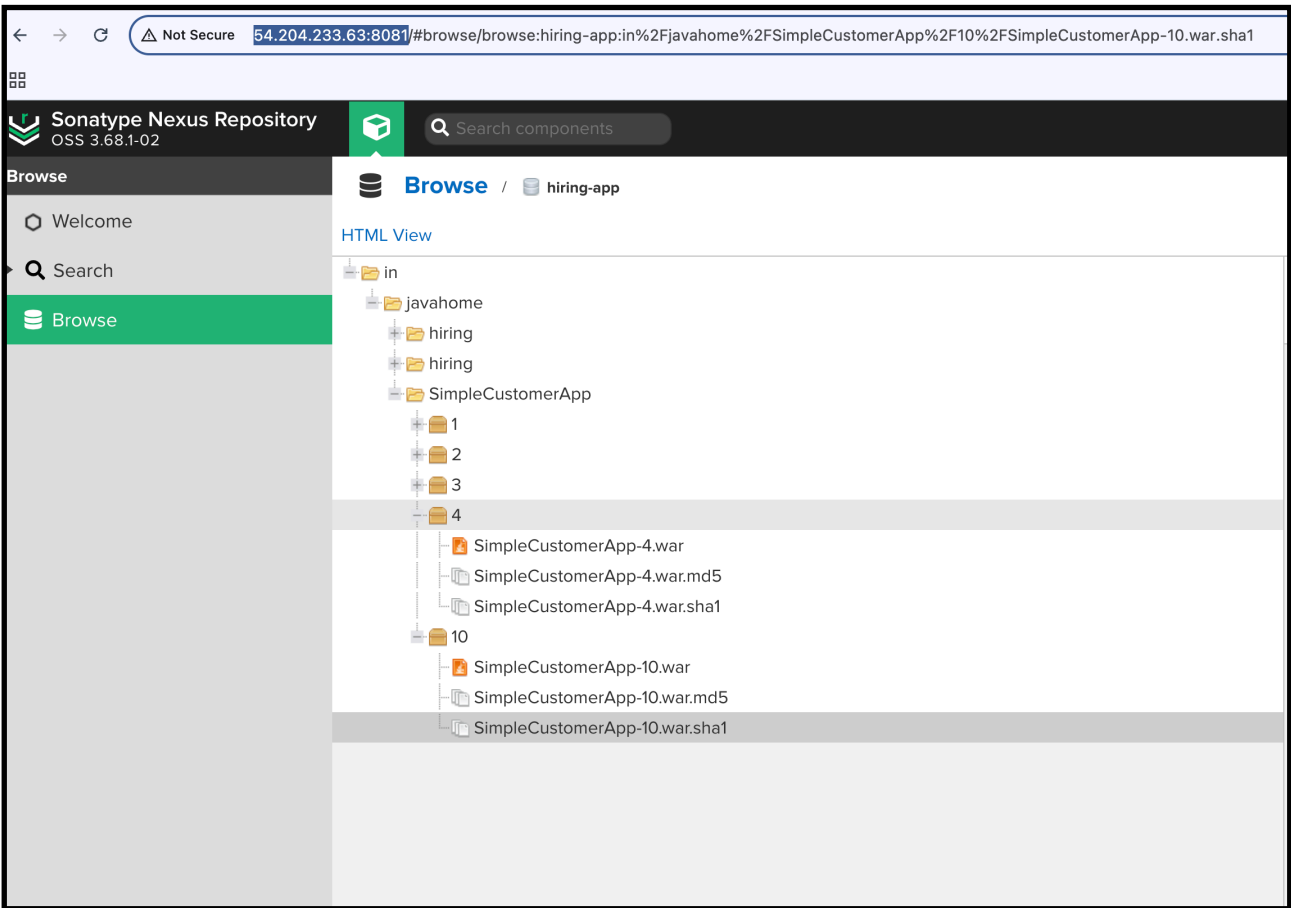
- > check inside the master Jenkins
- > cd /var/lib/jenkins/workspace/hiring-app-declarative

```
[root@ip-172-31-65-24 jenkins]# cd workspace/
[root@ip-172-31-65-24 workspace]# ls
first-job      hiring-app-declarative      job2      parametirized-job      pipeline      priv
hiring-app     hiring-app-declarative@tmp  job3      parametirized-job@tmp  pipeline@tmp  priv
[root@ip-172-31-65-24 workspace]# cd hiring-app-declarative
[root@ip-172-31-65-24 hiring-app-declarative]# ls
Jenkinsfile  README.md  Service-API.dockerfile  WebContent  XMLSave.txt  build  build.xml  po
[root@ip-172-31-65-24 hiring-app-declarative]#
```

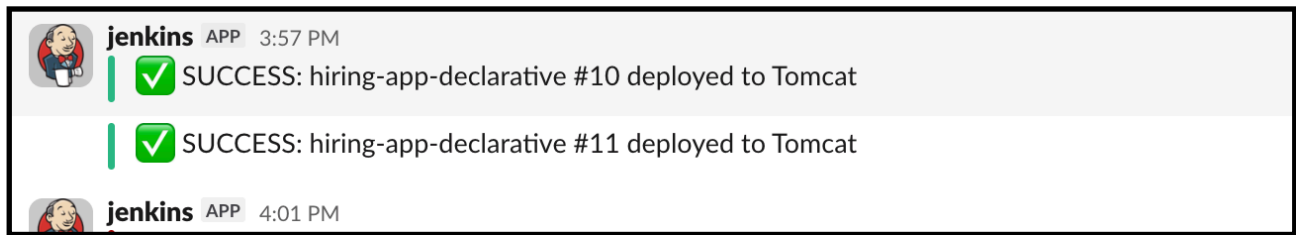
Sonarqube



Nexus



Slack notification:



2.Create one Scripted pipeline job

Scripted Pipeline Job

What it is

- **Fully Groovy-based**
- No fixed structure
- More control and flexibility
- Used for **complex logic**

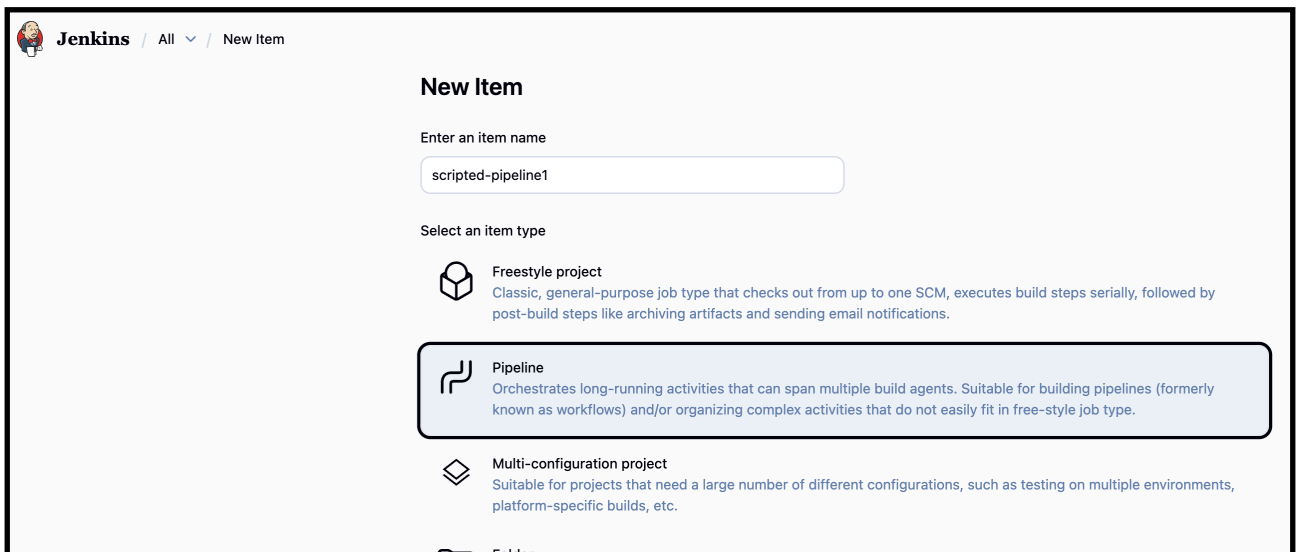
Key Characteristics

- Uses `node {}` instead of `pipeline {}`
- Can use loops, conditions, variables
- Harder to read & maintain

Steps

1. New Item → `scripted-pipeline1`
2. Select **Pipeline**
3. Paste Scripted pipeline code
4. Save → Build Now

Step 1



Jenkins / All / New Item

New Item

Enter an item name

Select an item type

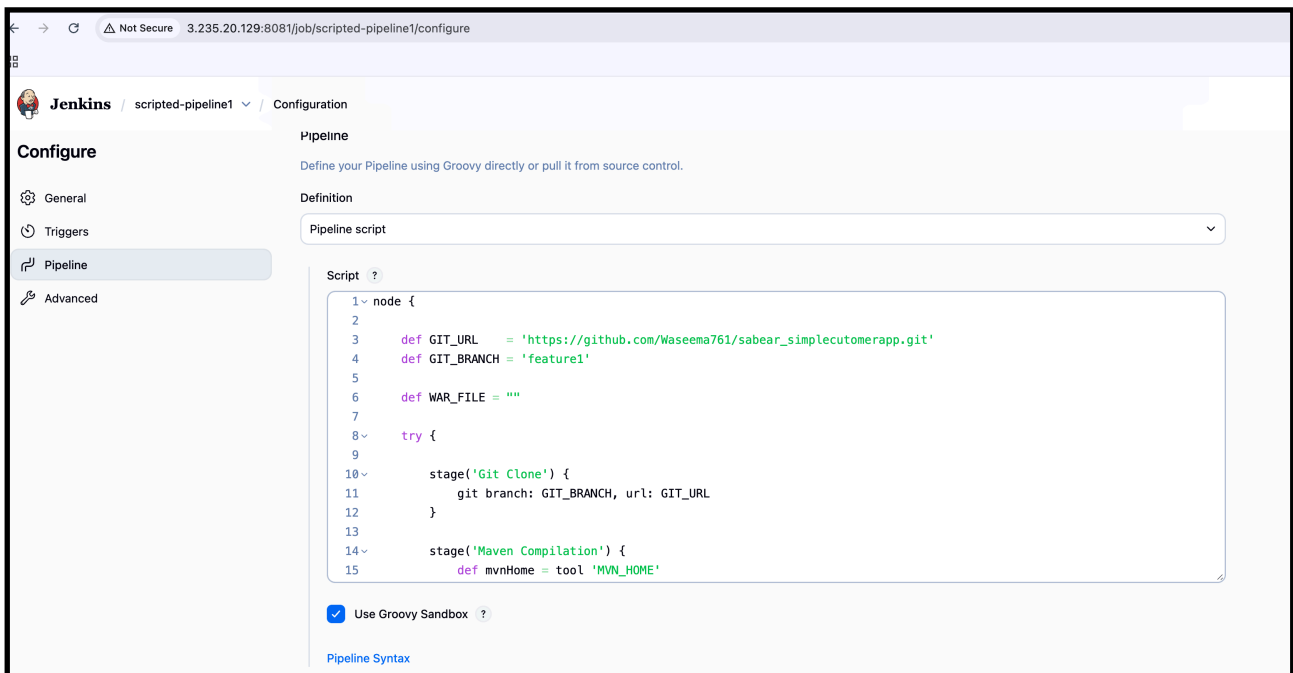
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

—-> now in job configuration add the pipeline code using groovy format

—-> and click build now and check status of the pipeline

—-> we are using scripting methods

Step 2



```
node {
```

```
    def GIT_URL    = 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
    def GIT_BRANCH = 'feature1'
```

```
    def WAR_FILE = ""
```

```
    try {
```

```
        stage('Git Clone') {
            git branch: GIT_BRANCH, url: GIT_URL
        }
```

```
        stage('Maven Compilation') {
            def mvnHome = tool 'MVN_HOME'
            sh """
            ${mvnHome}/bin/mvn clean install
```

```

    """
}

stage('Detect WAR File') {
    WAR_FILE = sh(
        script: "ls target/*.war | head -n 1",
        returnStdout: true
    ).trim()

    echo "Detected WAR: ${WAR_FILE}"
}

stage('SonarQube Integration') {
    def scannerHome = tool 'sonar_scanner'

    withSonarQubeEnv('sonarqube-server1') {
        sh """
        ${scannerHome}/bin/sonar-scanner \
        -Dsonar.projectKey=Sabear \
        -Dsonar.projectName=Sabear \
        -Dsonar.projectVersion=1.0 \
        -Dsonar.sources=src \
        -Dsonar.java.binaries=target/**/WEB-INF/classes
        """
    }
}

stage('Nexus Artifactory') {
    nexusArtifactUploader(
        nexusVersion: 'nexus3',
        protocol: 'http',
        nexusUrl: '54.204.233.63:8081',
        repository: 'hiring-app',

```

```
credentialsId: 'nexus-user',
groupId: 'in.javahome',
version: env.BUILD_NUMBER,
artifacts: [[
    artifactId: 'SimpleCustomerApp',
    classifier: '',
    file: WAR_FILE,
    type: 'war'
]]
)
}
```

```
stage('Deploy On Tomcat') {
    deploy adapters: [
        tomcat9(
            credentialsId: 'tomcat_credentials',
            url: 'http://98.92.64.216:8080'
        )
    ],
    contextPath: 'SimpleCustomerApp',
    war: WAR_FILE
}
```

```
stage('Slack Notification') {
    slackSend(
        channel: '#jenkins-integration',
        color: 'good',
        message: "✅ BUILD SUCCESS: $
{env.JOB_NAME} #${env.BUILD_NUMBER}"
    )
}
```

```

    } catch (err) {

        slackSend(
            channel: '#jenkins-integration',
            color: 'danger',
            message: "❌ BUILD FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
        )

        throw err
    }
}

```

Step 3

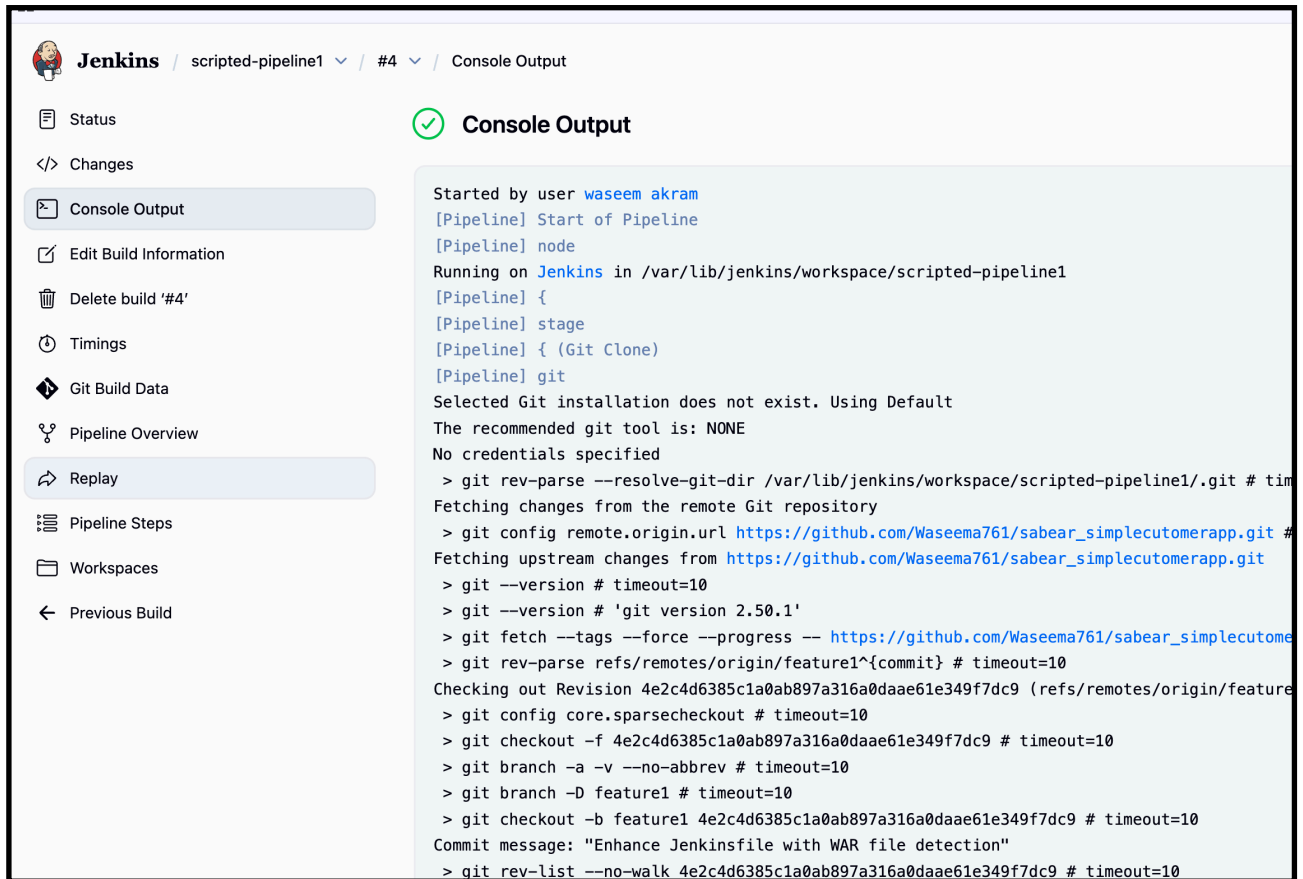
—> save the code and click build now

—> click build now

The screenshot shows the Jenkins web interface for a pipeline named "scripted-pipeline1". The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, SonarQube, Stages, Rename, and Pipeline Syntax. The main content area shows the pipeline status as "scripted-pipeline1" with a green checkmark icon. Below this, the "SonarQube Quality Gate" section shows a "Passed" status for "Sabear" and "Success" for "server-side processing". A "Permalinks" section lists several links for build history. At the bottom, a "Builds" table shows the last four builds, with build #1 failed and builds #2-4 successful.

Build Number	Status	Time
#4	Success	9:54 AM
#3	Success	9:46 AM
#2	Success	9:32 AM
#1	Failed	9:26 AM

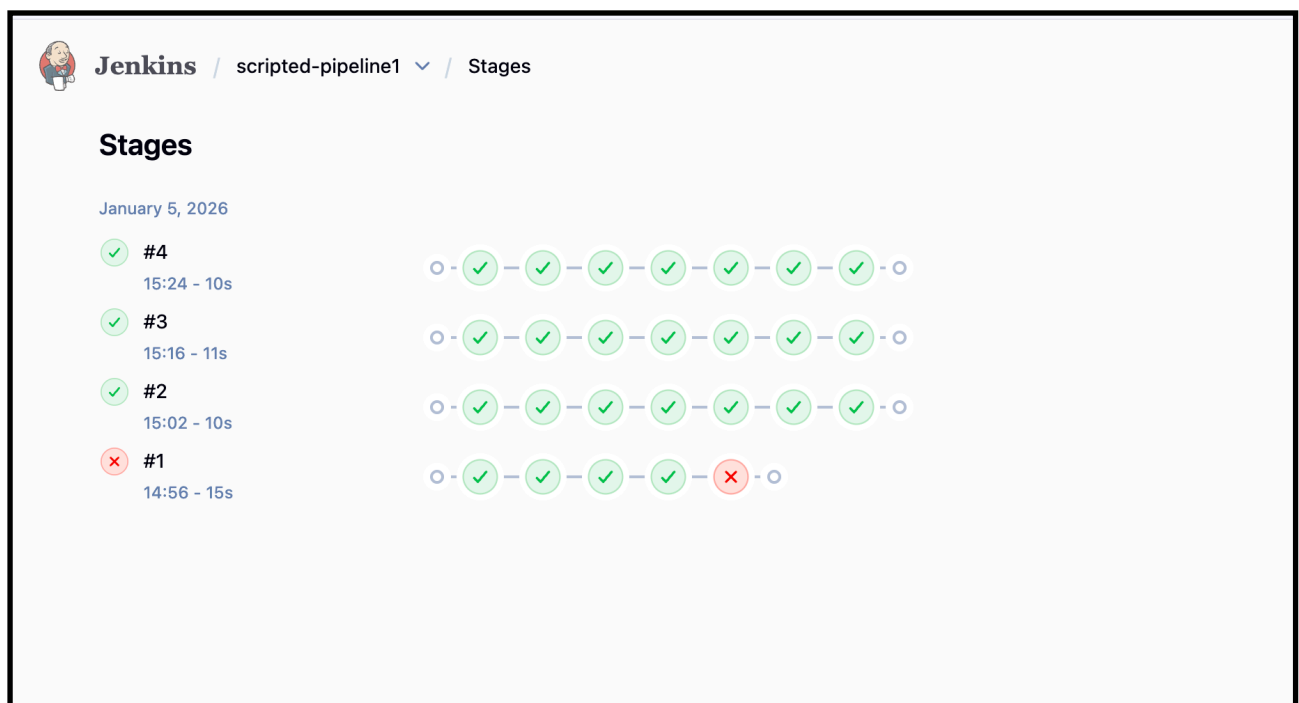
—> now check the console output



The screenshot shows the Jenkins 'Console Output' page for a build named 'scripted-pipeline1' at step '#4'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build '#4', Timings, Git Build Data, Pipeline Overview, Replay, Pipeline Steps, Workspaces, and Previous Build. The main area displays the console output, which shows the pipeline starting with 'node' and running 'git clone' on a repository. The output includes the following commands and their results:

```
Started by user waseem akram
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/scripted-pipeline1
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/scripted-pipeline1/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/sabear_simplecutomerapp.git # timeout=10
Fetching upstream changes from https://github.com/Waseema761/sabear_simplecutomerapp.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/sabear_simplecutomerapp.git # timeout=10
> git rev-parse refs/remotes/origin/feature1^{commit} # timeout=10
Checking out Revision 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 (refs/remotes/origin/feature1)
> git config core.sparsecheckout # timeout=10
> git checkout -f 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D feature1 # timeout=10
> git checkout -b feature1 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 # timeout=10
Commit message: "Enhance Jenkinsfile with WAR file detection"
> git rev-list --no-walk 4e2c4d6385c1a0ab897a316a0daae61e349f7dc9 # timeout=10
```

—> check the stages

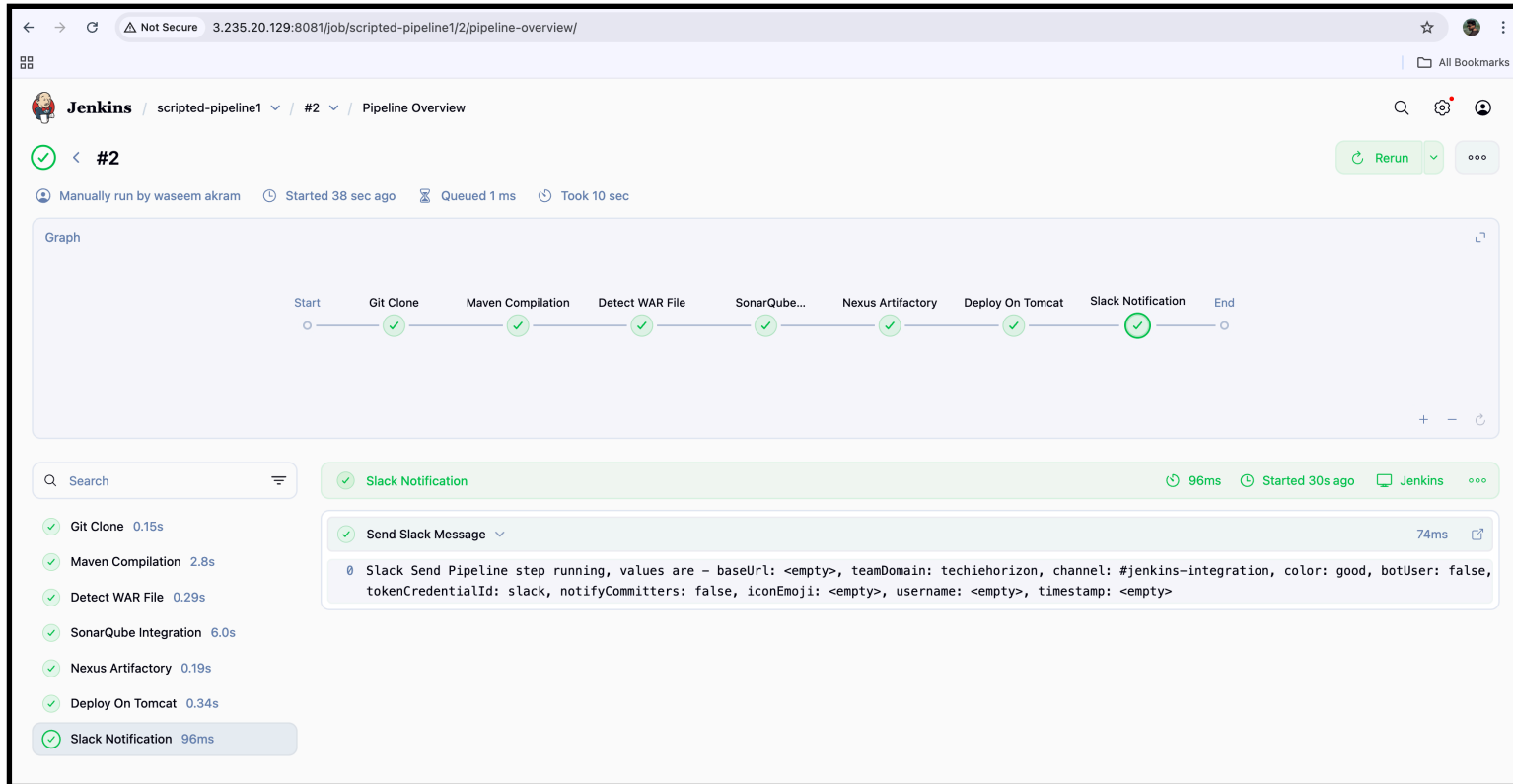


The screenshot shows the Jenkins 'Stages' page for the same build. The left sidebar is identical to the previous screenshot. The main area displays a list of stages with their status and duration:

Stage	Status	Duration
#4	Success	15:24 - 10s
#3	Success	15:16 - 11s
#2	Success	15:02 - 10s
#1	Failure	14:56 - 15s

Below the list, a visual representation of the stages is shown as a sequence of circles. The first three stages (#1, #2, #3) are marked with green checkmarks, indicating success. The fourth stage (#4) is marked with a red 'X', indicating failure.

—> check the pipeline overview



The screenshot displays the Jenkins Pipeline Overview page for a job named 'scripted-pipeline1'. The pipeline is currently at step #2, 'Slack Notification', which is marked as successful with a green checkmark. The pipeline graph shows a sequence of steps: Start, Git Clone, Maven Compilation, Detect WAR File, SonarQube Integration, Nexus Artifactory, Deploy On Tomcat, Slack Notification, and End. All steps are successful. The 'Slack Notification' step is expanded, showing its configuration: 'Send Slack Message' with a message body: '0 Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: techiehorizon, channel: #jenkins-integration, color: good, botUser: false, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>'. The step took 96ms to complete. The left sidebar lists the steps with their durations: Git Clone (0.15s), Maven Compilation (2.8s), Detect WAR File (0.29s), SonarQube Integration (6.0s), Nexus Artifactory (0.19s), Deploy On Tomcat (0.34s), and Slack Notification (96ms).

Jenkins / scripted-pipeline1 / #2 / Pipeline Overview

Manually run by waseem akram | Started 38 sec ago | Queued 1 ms | Took 10 sec

Graph

Start → Git Clone → Maven Compilation → Detect WAR File → SonarQube... → Nexus Artifactory → Deploy On Tomcat → Slack Notification → End

Search

Slack Notification 96ms | Started 30s ago | Jenkins

Send Slack Message 74ms

0 Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: techiehorizon, channel: #jenkins-integration, color: good, botUser: false, tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

Git Clone 0.15s

Maven Compilation 2.8s

Detect WAR File 0.29s

SonarQube Integration 6.0s

Nexus Artifactory 0.19s

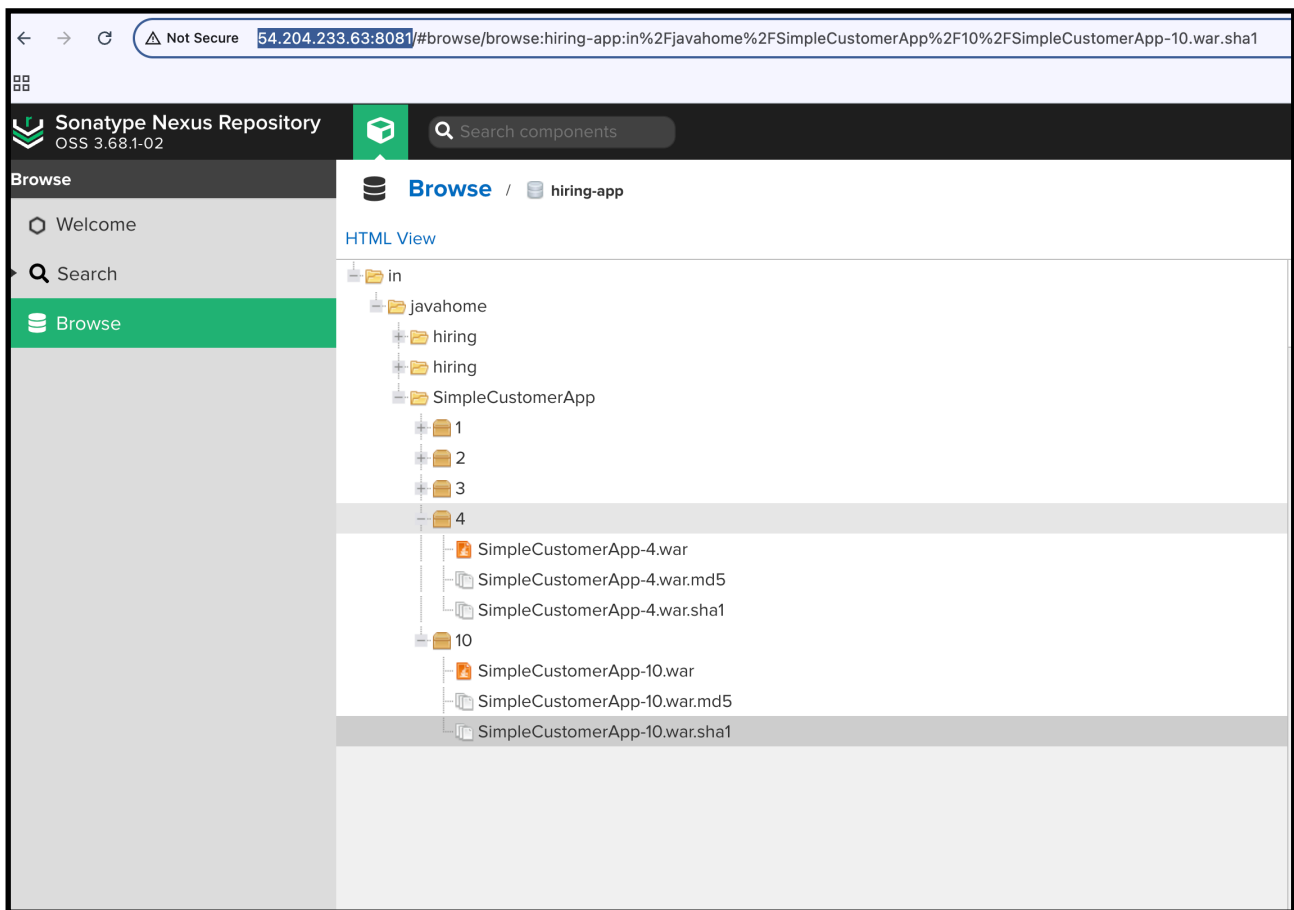
Deploy On Tomcat 0.34s

Slack Notification 96ms

Validate :

In nexus

We have builded 4 times and in nexus hiring-app repo
check for 4th file



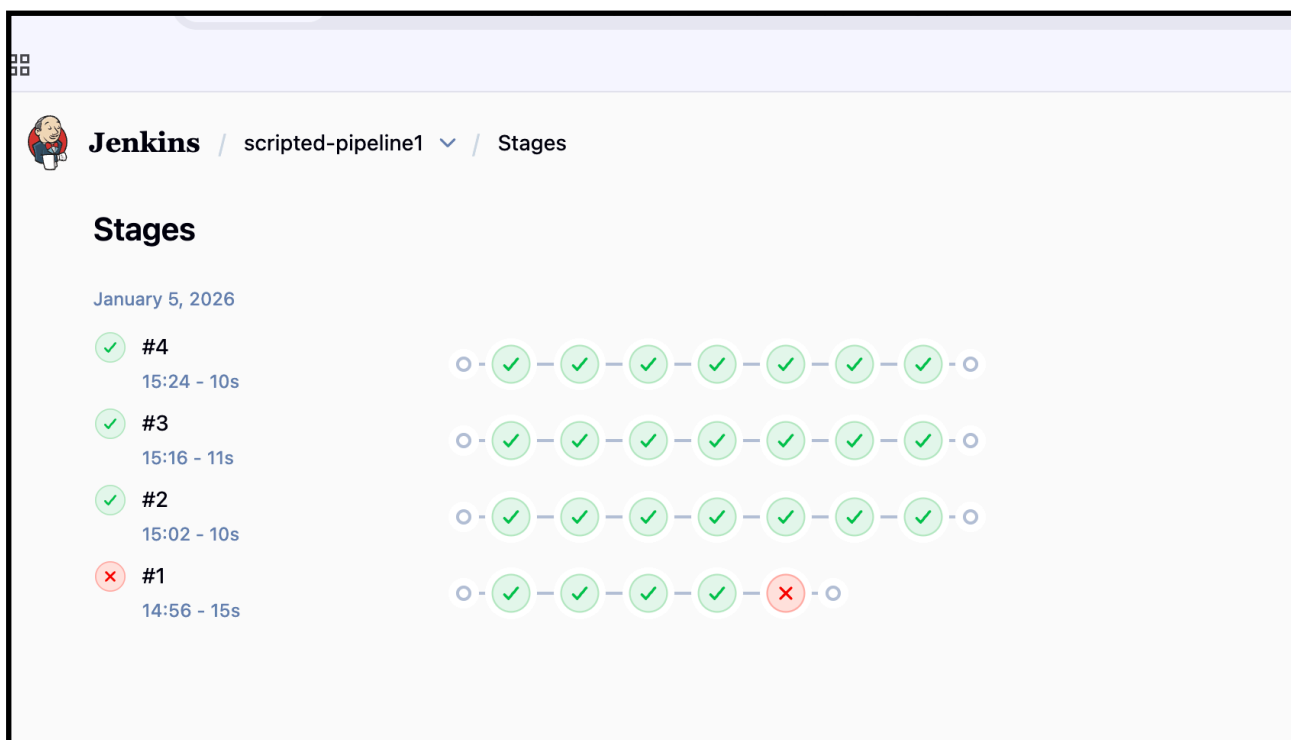
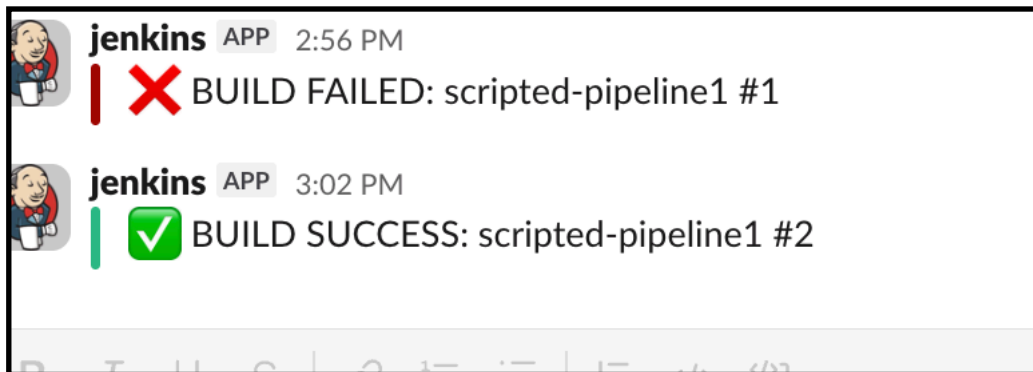
Git clone: validation in master jenkins

—> `cd /var/lib/jenkins/workspace/`

```
[root@ip-172-31-65-24 workspace]# cd scripted-pipeline1
[root@ip-172-31-65-24 scripted-pipeline1]# ls
Jenkinsfile  README.md  Service-API.dockerfile  WebContent  XMLSave.txt  build  build.xml  p
[root@ip-172-31-65-24 scripted-pipeline1]#
```

—> we are able to have the repo files

Slack notification :



3.Create one multi stage pipeline job

Objective:

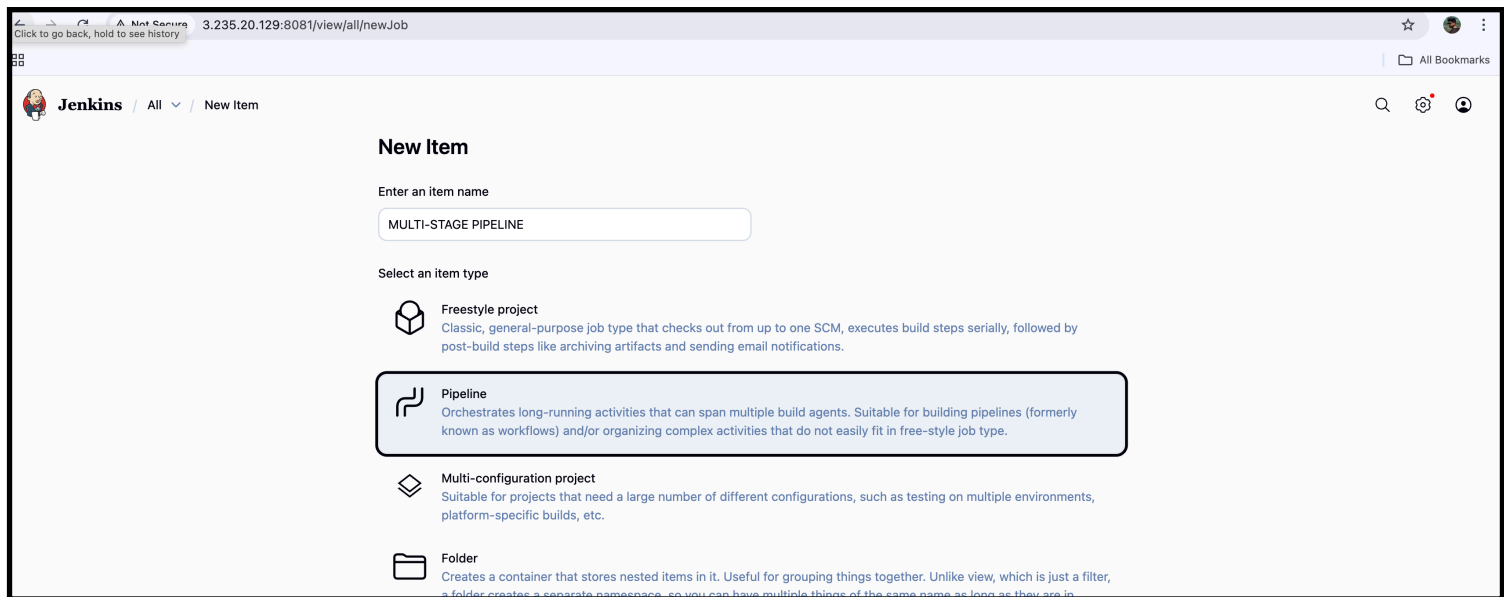
Create a Jenkins multi-stage pipeline to automate CI and CD using multiple sequential stages.

Step 1: Create Pipeline Job

1. Open **Jenkins Dashboard**
2. Click **New Item**
3. Enter job name:

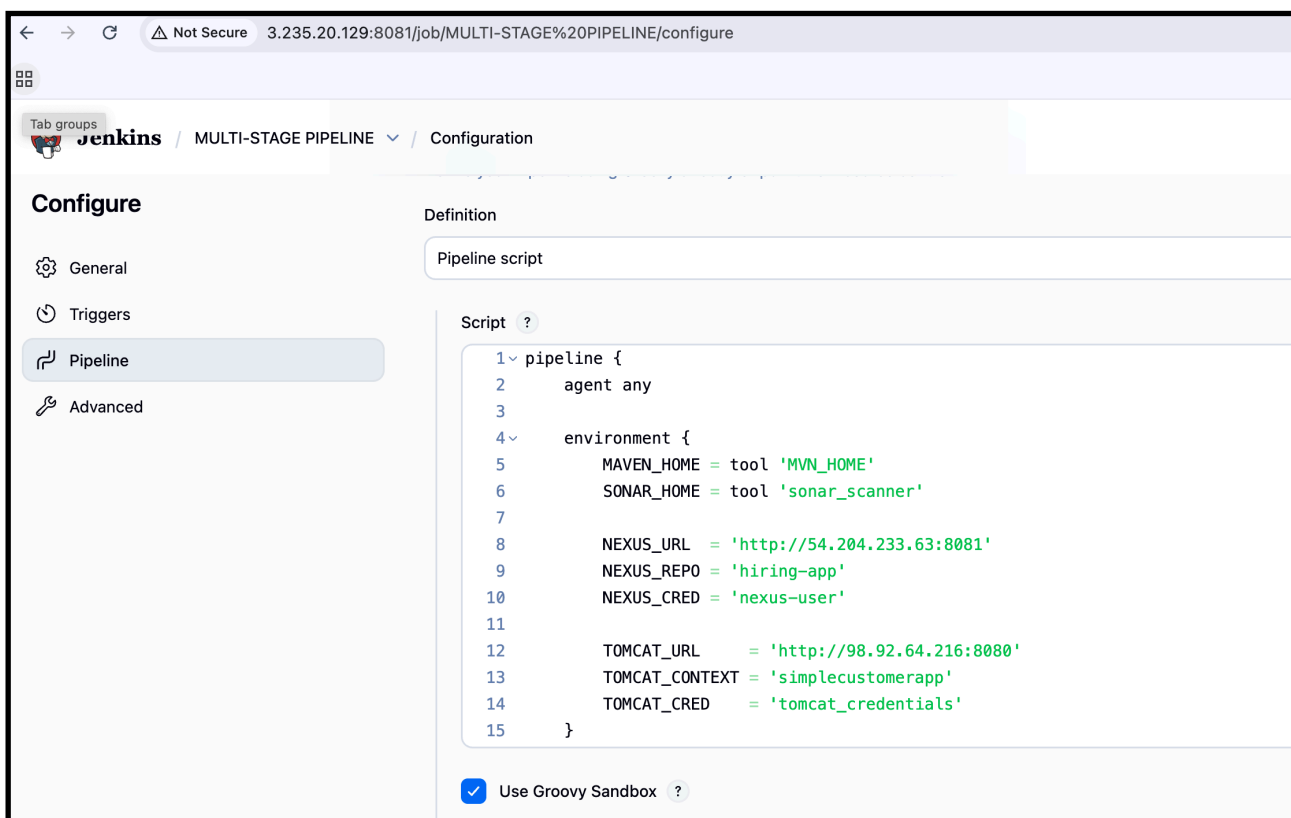
MULTI-STAGE PIPELINE

- 4.
5. Select **Pipeline**
6. Click **OK**



Step 2: Configure Pipeline

1. Scroll to **Pipeline** section
2. Select:
 - **Definition** → *Pipeline script*
3. Paste the **multi-stage pipeline code**
4. Click **Save**



Step 3: Multi-Stage Pipeline Flow

The pipeline is divided into the following stages:

Stage 1 – Git Clone

- Pulls source code from GitHub repository
- Ensures latest code is used for the build

◆ Stage 2 – Maven Build

- Compiles the application
- Generates the WAR file
- Verifies build success

◆ Stage 3 – SonarQube Analysis

- Performs static code analysis
- Checks code quality and bugs
- Sends analysis report to SonarQube server

◆ Stage 4 – Upload Artifact to Nexus

- Uploads the generated WAR file to Nexus repository
- Stores the artifact for versioning and reuse

◆ Stage 5 – Deploy to Tomcat

- Deploys the WAR file to Apache Tomcat server
- Makes the application available for access

◆ Stage 6 – Slack Notification

- Sends build status notification to Slack
- Notifies success or failure of the pipeline

```
pipeline {
```

```
    agent any
```

```
    environment {
```

```
        MAVEN_HOME = tool 'MVN_HOME'
```

```
        SONAR_HOME = tool 'sonar_scanner'
```

```
        NEXUS_URL = 'http://54.204.233.63:8081'
```

```
        NEXUS_REPO = 'hiring-app'
```

```
        NEXUS_CRED = 'nexus-user'
```

```
        TOMCAT_URL    = 'http://98.92.64.216:8080'
```

```
        TOMCAT_CONTEXT = 'simplecustomerapp'
```

```
        TOMCAT_CRED    = 'tomcat_credentials'
```

```
    }
```

```
    stages {
```

```
        stage('Git Clone') {
```

```
            steps {
```

```
                git 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
```

```
            }
```

```
        }
```

```
        stage('Build') {
```

```
            steps {
```

```
                sh "${MAVEN_HOME}/bin/mvn clean package"
```

```
            }
```

```
}
```

```
stage('Code Analysis') {
```

```
  steps {
```

```
    withSonarQubeEnv('sonarqube-server1') {
```

```
      sh """
```

```
        ${SONAR_HOME}/bin/sonar-scanner \
```

```
        -Dsonar.projectKey=Sabear \
```

```
        -Dsonar.projectName=Sabear \
```

```
        -Dsonar.sources=src \
```

```
        -Dsonar.java.binaries=target/**/WEB-INF/classes
```

```
      """
```

```
    }
```

```
  }
```

```
}
```

```
stage('Upload to Nexus') {
```

```
  steps {
```

```
    withCredentials([usernamePassword(
```

```
      credentialsId: "${NEXUS_CRED}",
```

```
      usernameVariable: 'NEXUS_USER',
```

```
      passwordVariable: 'NEXUS_PASS'
```

```
    )) {
```

```
      sh """
```

```
        WAR_FILE=$(ls target/*.war | head -n 1)
```

```
        curl -u ${NEXUS_USER}:${NEXUS_PASS} \
```

```
        --upload-file ${WAR_FILE} \
```

```
    ${NEXUS_URL}/repository/${NEXUS_REPO}/in/javahome/waseem/0.1/  
waseem-0.1.war
```

```
    """
```

```
    }
```

```
  }
```

```
}
```

```
stage('Deploy to Tomcat') {
```

```
  steps {
```

```
    withCredentials([usernamePassword(
```

```
      credentialsId: "${TOMCAT_CRED}",
```

```
      usernameVariable: 'TOMCAT_USER',
```

```
      passwordVariable: 'TOMCAT_PASS'
```

```
    )) {
```

```
      sh """
```

```
        WAR_FILE=$(ls target/*.war | head -n 1)
```

```
        curl -u ${TOMCAT_USER}:${TOMCAT_PASS} \
```

```
        -T ${WAR_FILE} \
```

```
        "${TOMCAT_URL}/manager/text/deploy?path=${TOMCAT_CONTEXT}  
&update=true"
```

```
      """
```

```
    }
```

```
  }
```

```
}
```

```
stage('Slack Notification') {
```

```
  steps {
```

```
    slackSend(
```

```
      channel: '#jenkins-integration',
```

```

        color: 'good',

        message: "✅ MULTI-STAGE PIPELINE SUCCESS: ${JOB_NAME} #${BUILD_NUMBER}"
    )
}

}

}

}

```

Step 4: Execute the Pipeline

1. Click **Build Now**
2. Monitor execution in **Stage View**
3. Ensure all stages execute sequentially

The screenshot displays the Jenkins web interface for a pipeline named 'MULTI-STAGE PIPELINE'. The browser address bar shows the URL '3.235.20.129:8081/job/MULTI-STAGE%20PIPELINE/'. The Jenkins logo and the pipeline name are at the top. On the left sidebar, the 'Build Now' button is highlighted with a black border. The main content area shows a green checkmark icon and the text 'MULTI-STAGE PIPELINE'. Below this, the 'SonarQube Quality Gate' section indicates 'Sabear Passed' and 'server-side processing: Success'. A 'Permalinks' section lists four links for the last build, stable build, successful build, and completed build, all from 12 minutes ago. At the bottom, a 'Builds' section shows a list of builds with a search filter and two builds listed: '#2 1:37 PM' and '#1 1:25 PM', both with green checkmarks.

--> check the console output

The screenshot shows the Jenkins web interface for a pipeline named 'MULTI-STAGE PIPELINE' at stage '#2'. The 'Console Output' tab is selected, displaying a log of the pipeline's execution. The log starts with 'Started by user waseem akram' and '[Pipeline] Start of Pipeline'. It then shows the execution of 'node' and 'tool' steps. The 'Git Clone' step is highlighted, showing the process of fetching changes from a remote repository, checking out a specific revision, and creating a new branch. The log includes the following commands and their outputs:

```
[Pipeline] {
[Pipeline] tool
[Pipeline] tool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/MULTI-STAGE PIPELINE
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/sabear_simplecuto
Fetching upstream changes from https://github.com/Waseema761/sabear_simplecuto
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/sabear_
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision f386c3ea540af6aff21dd0c7f2c55de7d31d5219 (refs/remotes/or
> git config core.sparsecheckout # timeout=10
> git checkout -f f386c3ea540af6aff21dd0c7f2c55de7d31d5219 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
```

The screenshot shows the Jenkins web interface for a pipeline named 'MULTI-STAGE PIPELINE' at stage '#2'. The 'Pipeline Overview' tab is selected, displaying a visual representation of the pipeline's execution. The pipeline is shown as a sequence of steps: Start, Git Clone, Build, Code Analysis, Upload to Nexus, Deploy to Tomcat, Slack Notification, and End. All steps are marked with green checkmarks, indicating successful completion. The 'Slack Notification' step is highlighted, showing its execution details:

```
0 Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: techiehorizon, channel: #jenkins-integration, color: good, botUser: fa
tokenCredentialId: slack, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
```

—-> check the stages these stages are multiple stages used in pipeline



Stage 1: git clone

Stage 2: build

Stage 3: code analysis

Stage 4: upload to nexus

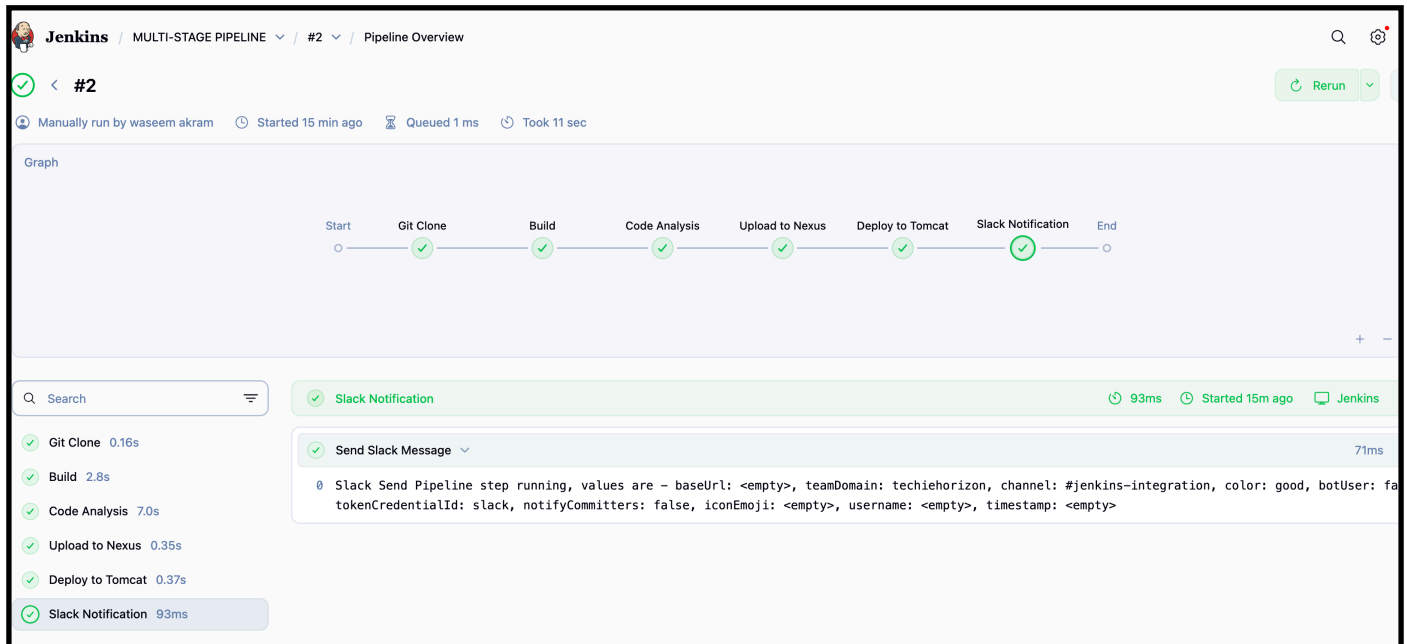
Stage 5: deploy to tomcat

Stage 6: slack notification

Step 5: Validate Multi-Stage Execution

- Each stage runs **one after another**
- Failure in any stage stops the next stages
- Stage View clearly shows **multiple stages**

Evidence for Task Submission:



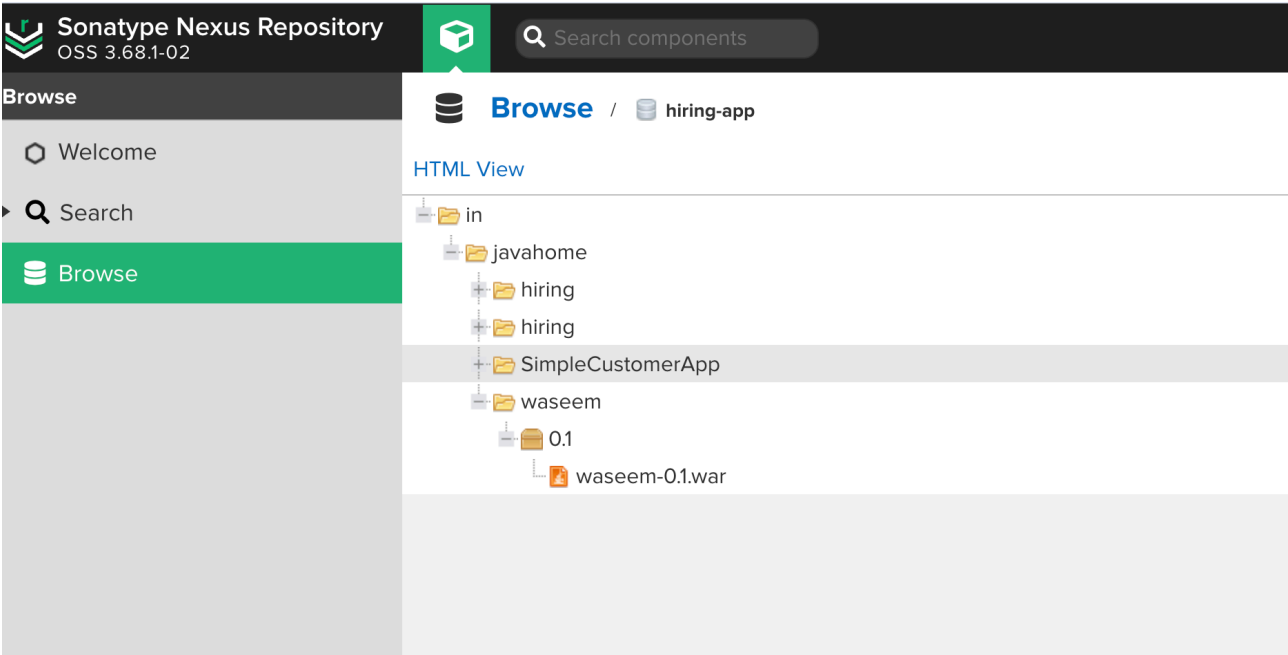
Description :

A Jenkins multi-stage pipeline was implemented to automate code checkout, build, quality analysis, artifact management, deployment, and notification in sequential stages.

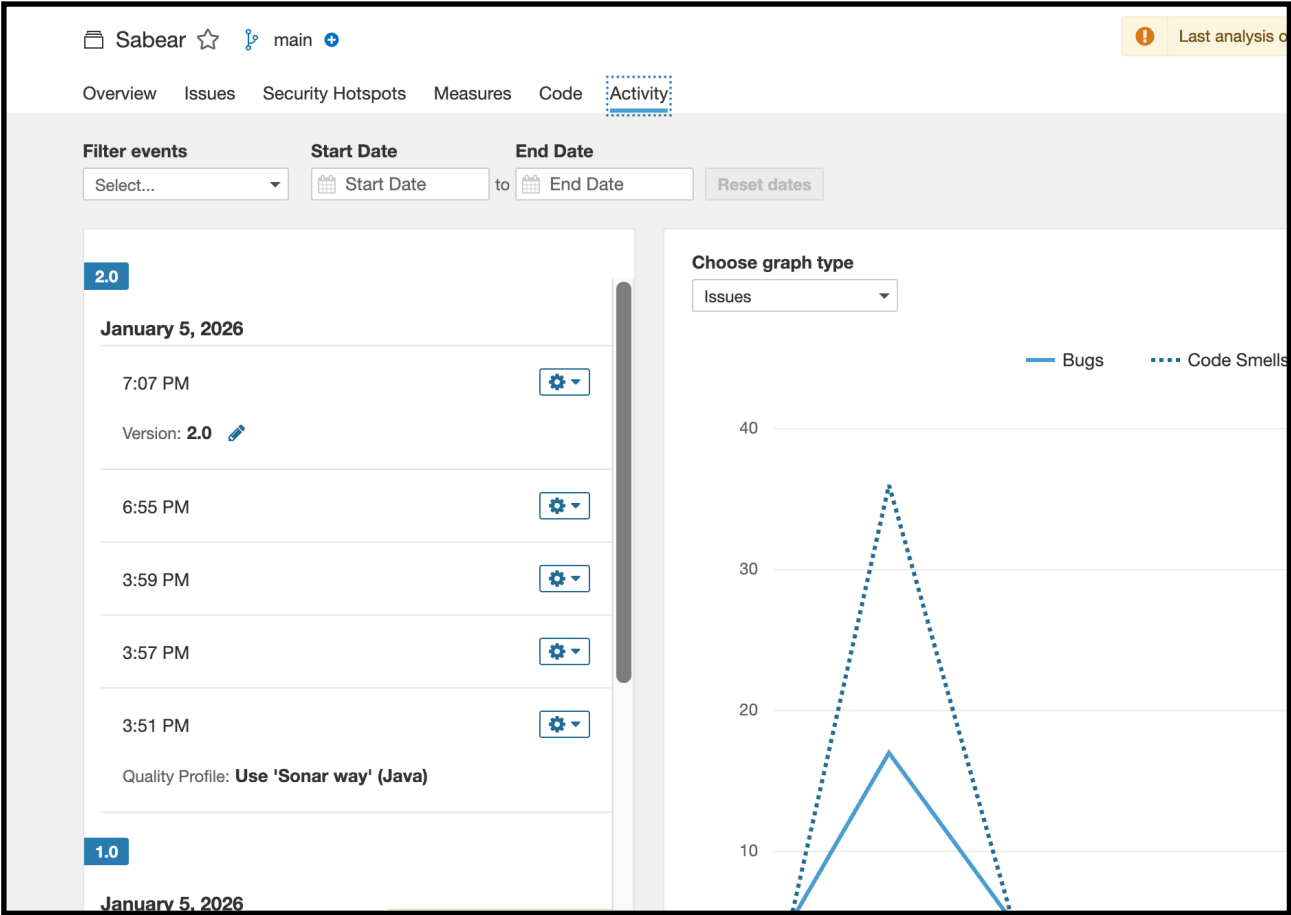
Final Checklist

- ✓ Pipeline job created
- ✓ Multiple stages configured
- ✓ CI and CD stages executed
- ✓ Nexus and Tomcat integrated
- ✓ Slack notification sent

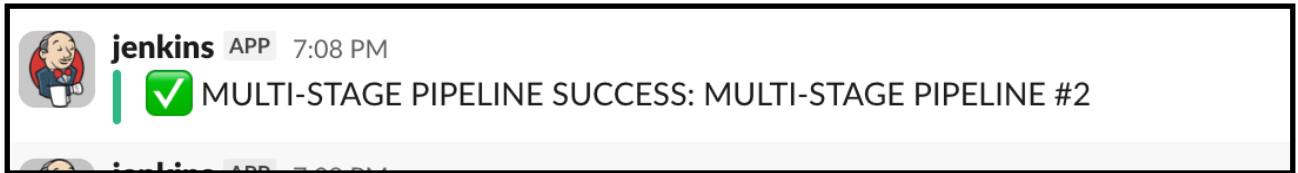
Nexus waseem name folder and sub directory created by the build



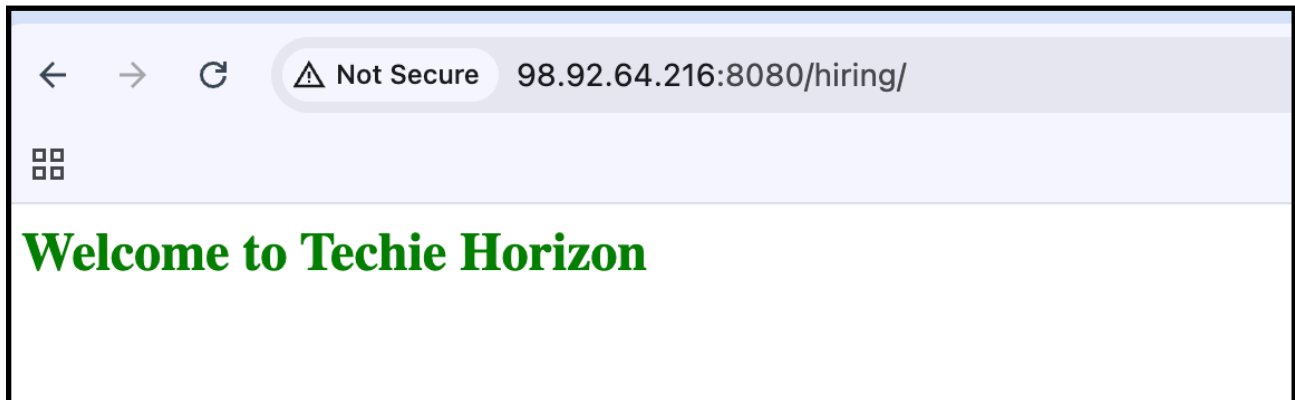
Sonarqube



Slack notification:



Tomcat :



4) Parallel Stage Pipeline Job

Jenkins Parallel CI/CD Pipeline – Execution Steps

Prerequisites:

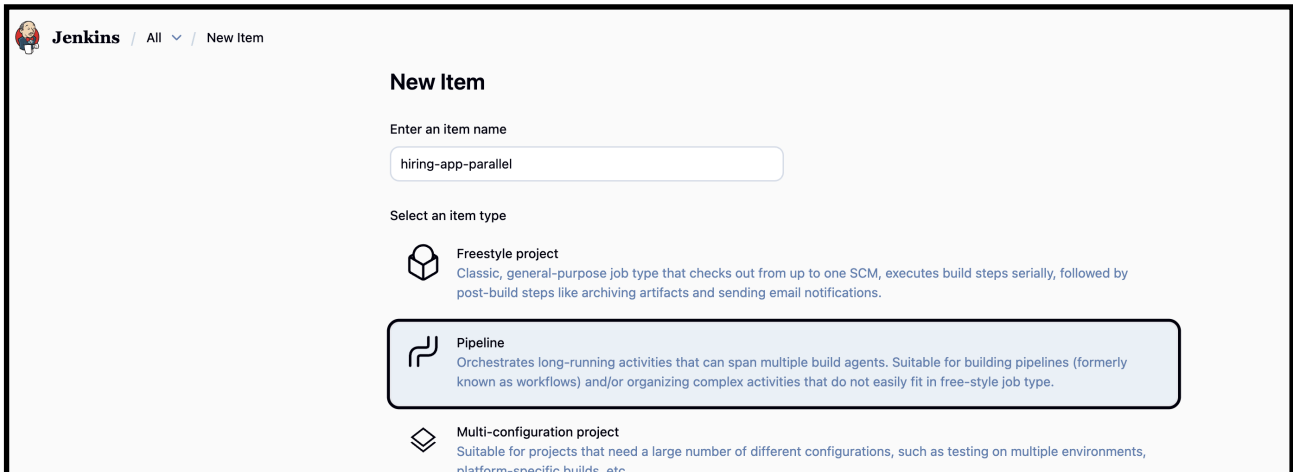
Before creating the pipeline, ensure the following are available:

- Jenkins installed and running
- Git installed on Jenkins server
- Maven installed and configured in Jenkins (MVN_HOME)
- SonarQube server running and reachable
- SonarQube Scanner installed (sonar_scanner)
- Nexus Repository Manager running
- Apache Tomcat server running with Manager access enabled
- Jenkins credentials configured:
 - Nexus username/password
 - Tomcat username/password
 - SonarQube authentication token
 - Slack token (optional)

-> Create a job :

Job name: hiring-app-parallel

—> select pipeline



Jenkins / All / New Item

New Item

Enter an item name

hiring-app-parallel

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Configure Tools in Jenkins

—>Maven Configuration

- Go to **Manage Jenkins** → **Global Tool Configuration**
- Add Maven:
 - Name: MVN_HOME
 - Install automatically or provide Maven path

—> SonarQube Scanner

- Add SonarQube Scanner:
 - Name: sonar_scanner
 - Path: /opt/sonar_scanner

—> SonarQube Server

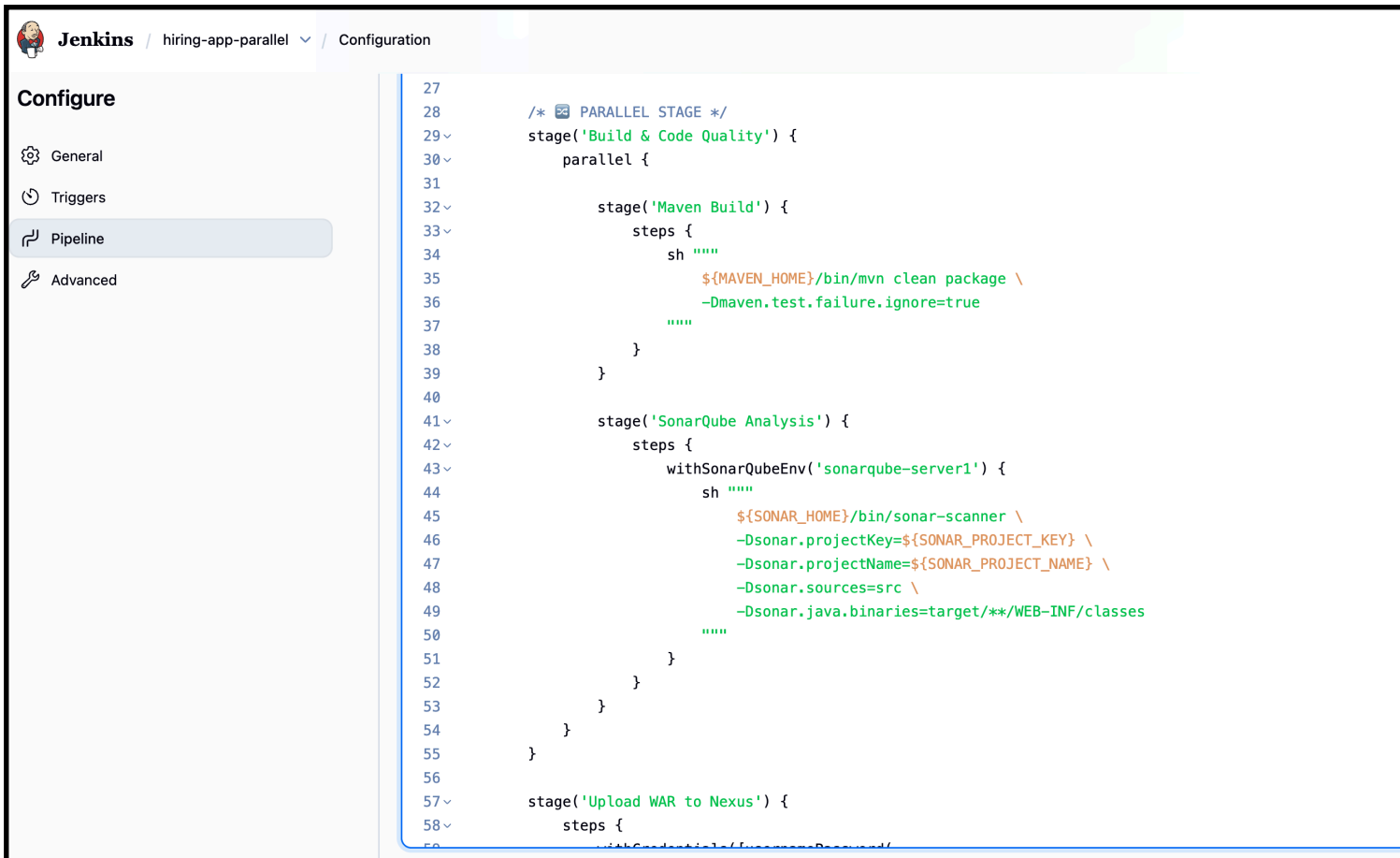
- Go to **Manage Jenkins** → **Configure System**
- Add SonarQube Server:
 - Name: sonarqube-server1
 - URL: http://<sonarqube-ip>:9000
 - Authentication Token (stored in Jenkins credentials)

Add Jenkins Credentials

Add the following credentials in **Manage Jenkins** → **Credentials**:

Purpose	Type	ID
Nexus	Username/Password	nexus-user
Tomcat	Username/Password	tomcat_credentials
SonarQube	Secret Text	Used in server config
Slack	Secret Text	slack

—> go to job configuration and add the code



The screenshot shows the Jenkins Configuration page for a job named 'hiring-app-parallel'. The 'Pipeline' tab is selected, displaying a Jenkinsfile script. The script defines a parallel stage with two sub-stages: 'Maven Build' and 'SonarQube Analysis'.

```
27
28  /* PARALLEL STAGE */
29  stage('Build & Code Quality') {
30    parallel {
31
32      stage('Maven Build') {
33        steps {
34          sh """
35            ${MAVEN_HOME}/bin/mvn clean package \
36            -Dmaven.test.failure.ignore=true
37          """
38        }
39      }
40
41      stage('SonarQube Analysis') {
42        steps {
43          withSonarQubeEnv('sonarqube-server1') {
44            sh """
45              ${SONAR_HOME}/bin/sonar-scanner \
46              -Dsonar.projectKey=${SONAR_PROJECT_KEY} \
47              -Dsonar.projectName=${SONAR_PROJECT_NAME} \
48              -Dsonar.sources=src \
49              -Dsonar.java.binaries=target/**/WEB-INF/classes
50            """
51          }
52        }
53      }
54    }
55  }
56
57  stage('Upload WAR to Nexus') {
58    steps {
59      withCredentials([usernamePassword(
```

Below pipeline I used for parallel use in pipeline :

```
pipeline {  
    agent any  
  
    environment {  
        MAVEN_HOME = tool 'MVN_HOME'  
        SONAR_HOME = tool 'sonar_scanner'  
  
        SONAR_PROJECT_KEY = 'Sabear'  
        SONAR_PROJECT_NAME = 'Sabear'  
  
        NEXUS_URL = 'http://54.204.233.63:8081'  
        NEXUS_REPO = 'hiring-app'  
        NEXUS_CRED = 'nexus-user'  
  
        TOMCAT_URL = 'http://98.92.64.216:8080'  
        TOMCAT_CONTEXT = 'simplecustomerapp'  
        TOMCAT_CRED = 'tomcat_credentials'  
    }  
  
    stages {
```

```
stage('Git Clone') {  
    steps {  
        git 'https://github.com/Waseema761/sabear_simplecutomerapp.git'  
    }  
}
```

/*  PARALLEL STAGE */

```
stage('Build & Code Quality') {  
    parallel {  
  
        stage('Maven Build') {  
            steps {  
                sh """  
                    ${MAVEN_HOME}/bin/mvn clean package \  
                    -Dmaven.test.failure.ignore=true  
                """  
            }  
        }  
    }  
}
```

```
stage('SonarQube Analysis') {  
    steps {  
        withSonarQubeEnv('sonarqube-server1') {  
            sh """  
                ${SONAR_HOME}/bin/sonar-scanner \  
                -Dsonar.projectKey=sonarqube-server1  
                -Dsonar.projectName=sonarqube-server1  
                -Dsonar.projectVersion=1.0  
                -Dsonar.language=java  
                -Dsonar.sourceEncoding=UTF-8  
            """  
        }  
    }  
}
```



```

        """
    }
}
}

```

```

stage('Deploy to Tomcat') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: "${TOMCAT_CRED}",
            usernameVariable: 'TOMCAT_USER',
            passwordVariable: 'TOMCAT_PASS'
        )]) {
            sh """
                WAR_FILE=$(ls target/*.war | head -n 1)

                curl -u ${TOMCAT_USER}:${TOMCAT_PASS} \
                -T ${WAR_FILE} \
                "${TOMCAT_URL}/manager/text/deploy?path=/${TOMCAT_CONTEXT}&update=true"
            """
        }
    }
}

```

```

stage('Slack Notification') {

```

```
steps {  
  script {  
    if (currentBuild.currentResult == 'SUCCESS') {  
      slackSend(  
        channel: '#jenkins-integration',  
        color: 'good',  
        message: "✅ SUCCESS: ${JOB_NAME} #${  
BUILD_NUMBER} deployed"  
      )  
    } else {  
      slackSend(  
        channel: '#jenkins-integration',  
        color: 'danger',  
        message: "❌ FAILED: ${JOB_NAME} #${  
BUILD_NUMBER}"  
      )  
    }  
  }  
}  
}  
  
post {  
  success {  
    echo "Pipeline completed successfully"
```

```
}  
  
failure {  
    echo "Pipeline failed"  
}  
  
}  
  
}
```

Pipeline Flow Explanation

—> Git Clone Stage

- Clones the application source code from GitHub repository.
- Ensures latest code is pulled for the build.

—>Parallel CI Stage – Build & Code Quality

This stage runs **two processes at the same time** to reduce build duration.

◆ Maven Build (Parallel)

- Executes Maven build command:
 - Cleans previous build
 - Packages application into WAR file
- Test failures do not stop the build.

◆ SonarQube Analysis (Parallel)

- Runs static code analysis using SonarQube Scanner
- Sends code quality report to SonarQube server
- Project appears on SonarQube dashboard after scan

—> Both stages run **in parallel**, improving pipeline performance.

—->Upload WAR to Nexus Repository

- WAR file generated in `target/` directory
- Artifact is uploaded to Nexus using REST API (`curl`)
- Authentication handled via Jenkins credentials
- Artifact stored in Nexus repository for versioning

—-> **Deploy Application to Tomcat**

- WAR file is deployed using **Tomcat Manager API**
- Uses Jenkins credentials for authentication
- Existing application is updated without downtime
- Application becomes accessible via browser after deployment

—-> **Slack Notification**

- Sends build status notification to Slack
- On success:
 - Displays job name and build number
- On failure:
 - Displays failure alert
- Helps teams track deployment status in real time

Post-Build Actions :

—> click build now and check the status of the code

Jenkins / hiring-app-parallel

hiring-app-parallel

SonarQube Quality Gate

Sabear **Passed**

server-side processing: **Success**

Permalinks

- [Last build \(#11\), 14 min ago](#)
- [Last stable build \(#11\), 14 min ago](#)
- [Last successful build \(#11\), 14 min ago](#)
- [Last failed build \(#10\), 17 min ago](#)
- [Last unsuccessful build \(#10\), 17 min ago](#)
- [Last completed build \(#11\), 14 min ago](#)

Builds

Filter

Today

✓	#11	2:42 PM	
✗	#10	2:39 PM	
✗	#9	2:38 PM	
✗	#8	2:38 PM	
✗	#7	2:32 PM	

On success:

- Displays confirmation message in Jenkins console

Validation :

—-> check the console output

← → ↻ ⚠ Not Secure 3.235.20.129:8081/job/hiring-app-parallel/11/console

Jenkins / hiring-app-parallel ▾ / #11 ▾ / Console Output

☰ Status ✔ **Console Output**

</> Changes

📁 Console Output

✍ Edit Build Information

🗑 Delete build '#11'

🕒 Timings

🔍 Git Build Data

🔗 Pipeline Overview

🔄 Restart from Stage

🔄 Replay

☰ Pipeline Steps

📁 Workspaces

← Previous Build

```
Started by user waseem akram
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/hiring-app-parallel
[Pipeline] {
[Pipeline] tool
[Pipeline] tool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/hiring-app-parallel/.git
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Waseema761/sabear_simplecutomerapp
Fetching upstream changes from https://github.com/Waseema761/sabear_simplecutomerapp.
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Waseema761/sabear_simple
```

—-> check the pipeline overview

Jenkins / hiring-app-parallel ▾ / #11 ▾ / Pipeline Overview

✔ < #11 🔄 Rerun ▾ ⋮

🕒 Manually run by waseem akram 🕒 Started 16 min ago ⌚ Queued 1 ms 🕒 Took 8.6 sec

Graph

```
graph LR
    Start((Start)) --> GitClone[Git Clone]
    GitClone --> BuildCodeQuality[Build & Code Quality]
    BuildCodeQuality --> UploadWAR[Upload WAR to Nexus]
    UploadWAR --> DeployTomcat[Deploy to Tomcat]
    DeployTomcat --> SlackNotification[Slack Notification]
    SlackNotification --> PostActions[Post Actions]
    PostActions --> End((End))
    BuildCodeQuality --> MavenBuild[Maven Build]
    BuildCodeQuality --> SonarQubeAnalysis[SonarQube Analysis]
```

🔍 Search

✔ Git Clone 0.29s

✔ Build & Code Quality 6.8s

✔ Upload WAR to Nexus 0.35s

✔ Deploy to Tomcat 0.35s

✔ Slack Notification 0.14s

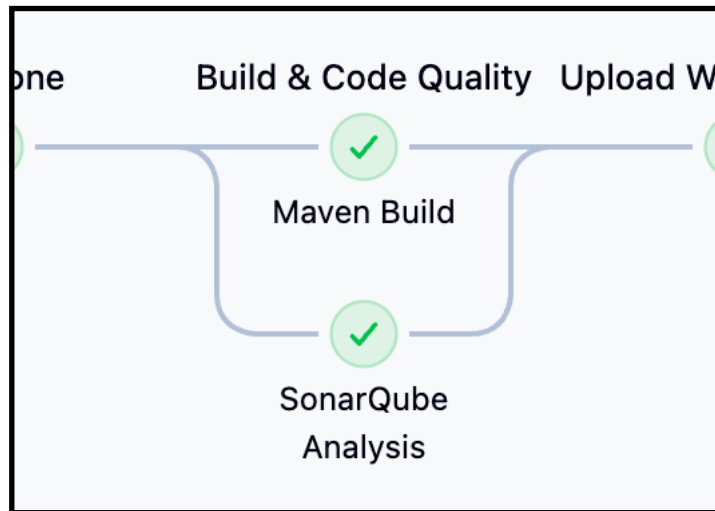
✔ Post Actions 35ms

✔ Post Actions 35ms Started 16m ago Jenkins ⋮

✔ Pipeline completed successfully ▾ 13ms

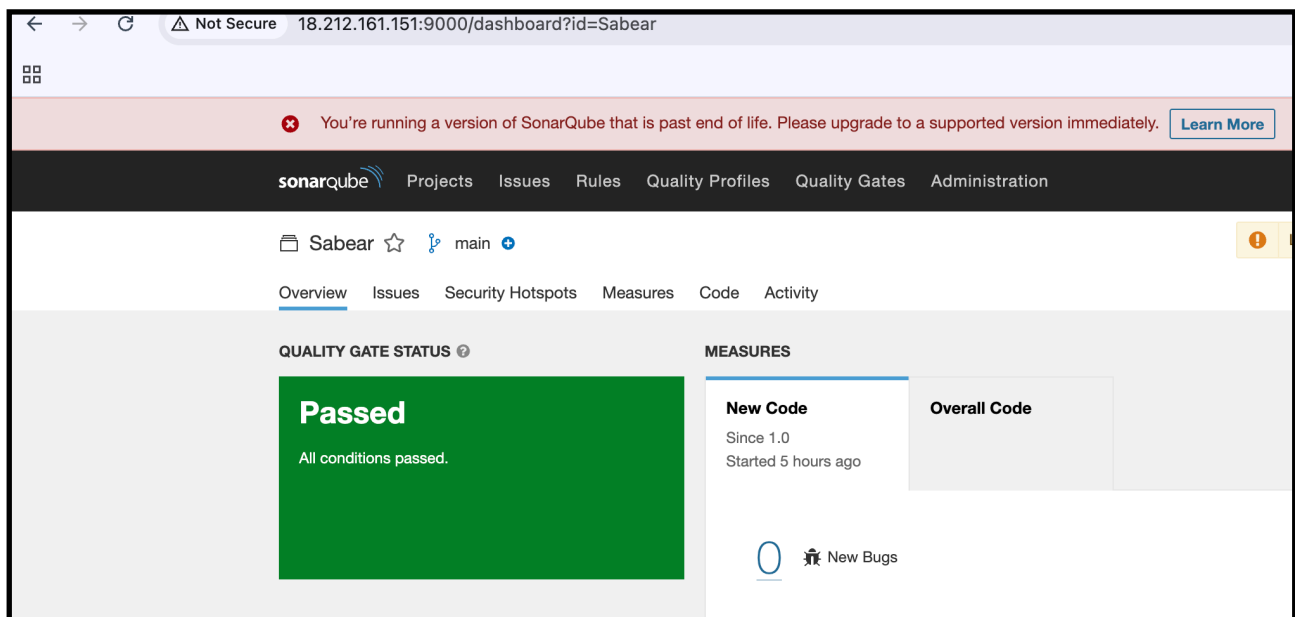
📄 Pipeline completed successfully

—-> we are able to see there is a parallel pipeline after git clone

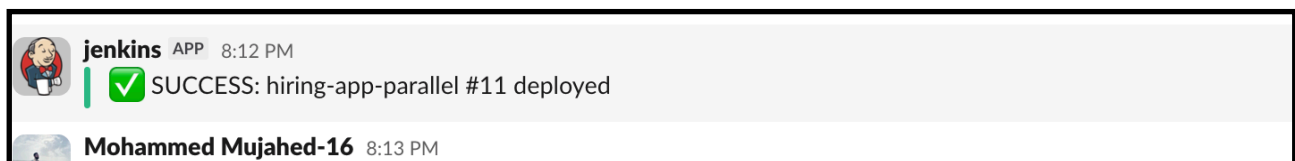


—> sonarqube :

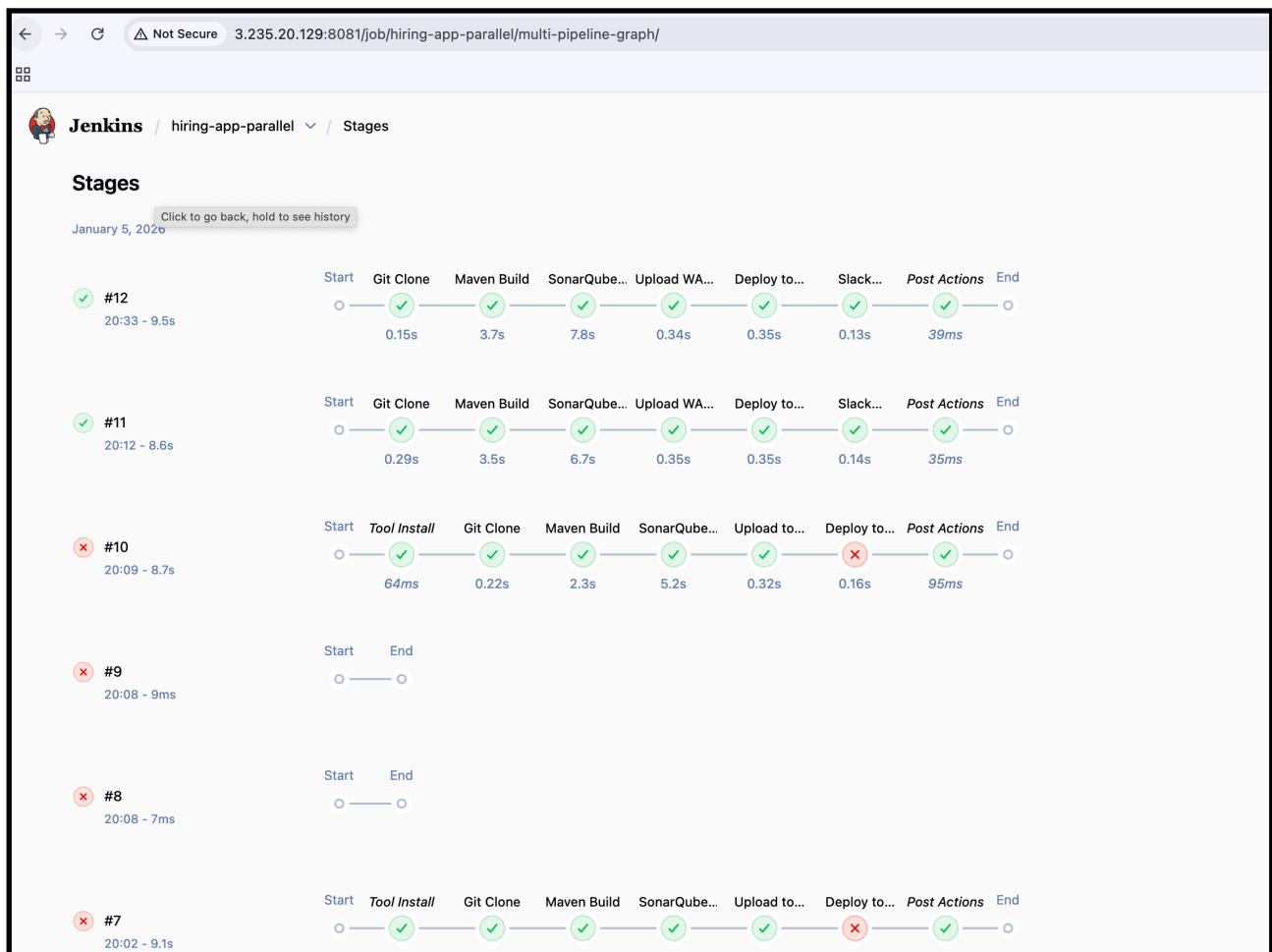
The screenshot shows the SonarQube web interface. The browser address bar displays '18.212.161.151:9000/project/activity?id=Sabear'. A red warning banner at the top states: 'You're running a version of SonarQube that is past end of life. Please upgrade to a supported version imm'. The navigation bar includes the SonarQube logo and links for 'Projects', 'Issues', 'Rules', 'Quality Profiles', 'Quality Gates', and 'Administration'. The main content area shows the project 'Sabear' with a star icon and a 'main' branch. The 'Activity' tab is selected, showing a list of events. The first event is dated 'January 5, 2026' at '8:12 PM'. It indicates the 'Quality Profile: Use 'Sonar way' (Java)' and 'Version: 2.0'. A 'Choose graph type' dropdown menu is set to 'Issues'. A line graph is partially visible at the bottom right, showing a peak and then a decline.



—-> slack notification:



—-> stages



Final Outcome

- Source code successfully built
- Code quality analysis completed
- Artifact stored in Nexus
- Application deployed to Tomcat
- Notifications sent to Slack
- Parallel execution reduced pipeline execution time

Conclusion

This parallel Jenkins pipeline demonstrates:

- Efficient CI using parallel stages

- Secure credential handling
- Automated artifact management
- Automated deployment
- Real-time notifications

This pipeline follows **real-world CI/CD best practices**.